

Relazione: Caratterizzazione e Controllo di
Motori Elettrici tramite Raspberry Pi e
Adafruit Motor HAT

Gruppo 100

Lorenzo Allocco
– Mat. 061270XXXX

Francesco Grimaldi
– Mat.0612708839

Riccardo La Porta
– Mat. 0612709691

Alessandro Altieri
– Mat.0612709239

5 Marzo 2026

Indice

1	Obiettivi dell'esperienza	3
2	Introduzione e cenni teorici	4
2.1	Il Ponte ad H (H-Bridge)	4
2.2	Modulazione PWM e Controller Dedicato PCA9685	5
2.3	Il Motore Passo-Passo (Stepper) e Stili di Avanzamento	7
3	Strumentazione e Componenti	8
4	Configurazione Hardware e Procedura Sperimentale	9
4.1	Connessioni Elettriche	9
4.2	Procedura di Controllo Software (Python 3)	9
5	Raccolta e Analisi dei Dati Sperimentali	10
5.1	Dati Rilevati sul Motore DC	10
5.2	Dati Rilevati sul Motore Stepper	10
6	Discussione dei Risultati	11
6.1	Comportamento Dinamico del Motore DC	11
6.2	Valutazione delle Modalità di Stepping del Motore Passo-Passo	11
7	Conclusioni	11

1 Obiettivi dell'esperienza

La presente relazione ha l'obiettivo di analizzare e descrivere il principio di funzionamento e le modalità di controllo digitale di due differenti tipologie di attuatori elettrici: un motore in corrente continua a magneti permanenti e un motore passo-passo (Stepper) bipolare. Nello specifico, l'esperienza si è focalizzata sull'impiego di un sistema basato su piattaforma **Raspberry Pi** accoppiato alla scheda di espansione **Adafruit DC and Stepper Motor HAT**, approfondendo i seguenti aspetti operativi:

- **Motore DC:** determinazione sperimentale della caratteristica tensione-velocità a vuoto mediante la modulazione della larghezza di impulso (PWM).
- **Motore Stepper:** studio delle sequenze di commutazione e implementazione delle logiche di stepping (Single, Double, Interleaved e Microstepping) per la verifica della precisione dell'angolo di passo e della fluidità del movimento.

2 Introduzione e cenni teorici

Il motore a corrente continua (DC) converte l'energia elettrica in ingresso in energia meccanica rotazionale basandosi sulla legge della forza di Laplace:

$$F = I(L \cdot B)$$

All'equilibrio elettrico di regime, la tensione d'armatura V_a applicata ai morsetti è bilanciata dalla caduta sulla resistenza interna e dalla forza contro-elettromotrice E indotta dal movimento del rotore:

$$V_a = R_a I_a + E$$

Dato che $E = K_e \cdot \omega$ (dove K_e è la costante di forza contro-elettromotrice e ω la velocità angolare), in condizioni di funzionamento a vuoto la corrente I_a assorbita assume valori minimi trascurabili, rendendo la velocità di rotazione direttamente proporzionale alla tensione applicata: $V_a \approx K_e \cdot \omega$.

2.1 Il Ponte ad H (H-Bridge)

Per invertire il verso di rotazione di un motore in corrente continua è necessario invertire la polarità della tensione ai suoi capi. Il circuito classico utilizzato a questo scopo è il **ponte ad H**, così chiamato per la disposizione dei componenti che ricorda la lettera "H".

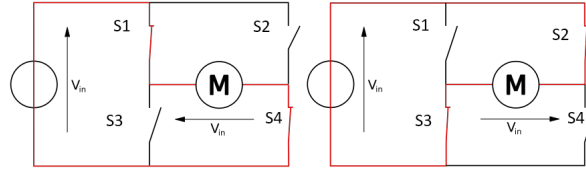


Figura 1: Configurazione del motore con ponte ad H. Inversione di polarità ai capi del motore.

Il circuito è costituito da quattro interruttori elettronici disposti in coppia su due rami verticali, con il motore posizionato sul ramo centrale trasversale, come in Figura 1. Nell'hardware Adafruit impiegato, tali interruttori sono costituiti da MOSFET di potenza integrati nel chipset **TB6612**. Il funzionamento si articola in quattro stati logici fondamentali:

1. **Marcia Avanti:** Chiudendo l'interruttore in alto a sinistra (S1) e quello in basso a destra (S4), la corrente attraversa il motore da sinistra a destra.
2. **Marcia Indietro:** Chiudendo l'interruttore in alto a destra (S2) e quello in basso a sinistra (S3), la polarità si inverte e la corrente scorre da destra a sinistra.

3. **Frenatura Dinamica (Brake):** Chiudendo simultaneamente i due interruttori inferiori (S3 e S4) o i due superiori (S1 e S2), i morsetti del motore vengono cortocircuitati tra loro. L'energia cinetica residua viene dissipata sotto forma di corrente elettrica sul corto, arrestando bruscamente l'albero.
4. **Rilascio (Coast):** Aprendo tutti e quattro gli interruttori, il motore viene isolato elettricamente dalla sorgente e continua a girare per inerzia fino al completo arresto dovuto agli attriti meccanici.

Nota di sicurezza: La chiusura contemporanea degli interruttori sullo stesso ramo verticale (es. S1 e S3) causerebbe un cortocircuito diretto della linea di alimentazione verso massa (*shoot-through*), danneggiando i componenti. Il chip TB6612 integra internamente una logica di blocco hardware per impedire questa condizione.

2.2 Modulazione PWM e Controller Dedicato PCA9685

Per variare la velocità del motore senza dissipare potenza in calore su componenti resistivi, si parzializza l'energia d'alimentazione tramite un segnale ad onda quadra ad alta frequenza denominato **PWM** (Pulse Width Modulation). Il Duty Cycle (D) rappresenta la frazione di tempo in cui il segnale è alto (T_{on}) rispetto al periodo totale (T):

$$D = \frac{T_{\text{on}}}{T} \cdot 100\%$$

La tensione media equivalente applicata sarà pari a: $V_{\text{media}} = D \cdot V_{\text{max}}$.

Poiché la CPU del Raspberry Pi non possiede un numero sufficiente di timer hardware per generare molteplici canali PWM indipendenti, l'Adafruit HAT introduce a bordo un chip dedicato: il **PCA9685**, mostrato in figura 2. Questo controller comunica con il Raspberry Pi tramite bus seriale **I2C** (usando solo i pin SDA e SCL) e genera autonomamente i segnali PWM necessari a pilotare i ponti ad H (contrassegnati dalla figura 2 sottostante dal colore verde) con una risoluzione a 8-bit (circa 256 livelli di velocità stabili, pari a incrementi dello 0.5%).

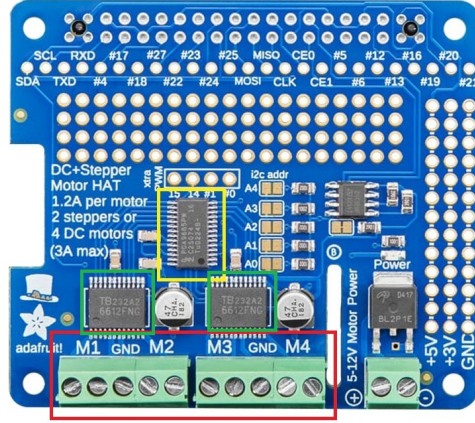


Figura 2: Motor HAT per Raspberry Pi. Evidenziato in giallo il modulo PCA9685.

Il collegamento del motore in corrente continua (DC) alla scheda di espansione avviene tramite una delle quattro morsettiere a vite dedicate, contrassegnate nell'immagine in rosso, dalle sigle da M1 a M4 (ad esempio, i due terminali del motore vengono inseriti nei due contatti di M1). Poiché il motore DC a spazzole non ha una polarità rigida, l'ordine dei fili non è vincolante: un'eventuale inversione determinerà semplicemente l'inversione del verso di rotazione iniziale, parametro comunque modificabile via software tramite il segno del comando *throttle*. È fondamentale, inoltre, collegare una sorgente di alimentazione esterna (es. 12VDC) alla morsettieria a vite a 2 pin dedicata ai motori presente sul HAT, rispettando la polarità positiva e negativa. Questo passaggio è cruciale per separare la corrente di potenza richiesta dai motori dall'alimentazione logica del Raspberry Pi, evitando pericolosi ritorni di corrente o cali di tensione sul microcontrollore.

2.3 Il Motore Passo-Passo (Stepper) e Stili di Avanzamento

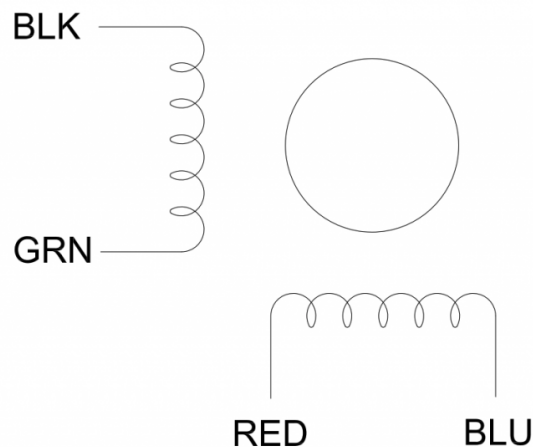


Figura 3: Schema semplificato della disposizione delle bobine in un motore stepper bipolare.

Il motore passo-passo si muove per scatti discreti azionando in sequenza temporale i campi magnetici delle sue bobine interne, disposte come in figura 3. L'accoppiamento tra il Raspberry Pi e l'HAT consente di sfruttare quattro modalità di stepping software native:

- **Single Step** (`stepper.SINGLE`): Viene alimentata una sola bobina alla volta. Rappresenta la modalità a minor consumo energetico, ma esprime la coppia di mantenimento più bassa.
- **Double Step** (`stepper.DOUBLE`): Vengono mantenute attive due bobine simultaneamente ad ogni passo. Il consumo di corrente raddoppia, ma la coppia motrice aumenta di circa il 25%.
- **Interleaved Step** (`stepper.INTERLEAVE`): Alterna passi a bobina singola a passi a doppia bobina. Questa configurazione implementa l'avanzamento ad **Half-Step** (mezzo passo), dimezzando l'angolo geometrico fondamentale e raddoppiando la risoluzione dello spostamento.
- **Microstepping** (`stepper.MICROSTEP`): Il controller PCA9685 modula la corrente nelle bobine tramite segnali PWM sinusoidali cooperanti. La transizione tra i passi non è più netta ma graduale, massimizzando la fluidità dinamica e minimizzando le risonanze meccaniche.

3 Strumentazione e Componenti

L'architettura hardware predisposta per l'esecuzione dell'esperienza comprende i seguenti elementi:

Componente / Strumento	Modello	Caratteristiche Principali
Controllore	Raspberry Pi Model 3B+	Single Board Computer
Interfaccia di Potenza	Adafruit Motor HAT	Integra 4 Ponti ad H (TB6612) e driver PWM I2C (PCA9685)
Motore DC	Adafruit Toy / Hobby Motor	Motore a spazzole e magneti permanenti
Motore Passo-Passo	NEMA Bipolare	4 fili, 200 passi/giro (Angolo base di 1.8°)
Alimentatore Esterno	Standard da banco 0-30V	Regolato a 12VDC per l'alimentazione dei motori
Strumento di Analisi	Oscilloscopio Hantek 60000	2 canali, utilizzato per monitorare le uscite dei driver
Multimetro digitale	/	Misurazione dei livelli di tensione e delle polarità erogati per i motori elettrici

Tabella 1: Specifiche della strumentazione e dei componenti hardware impiegati.

4 Configurazione Hardware e Procedura Sperimentale

4.1 Connessioni Elettriche

L'Adafruit Motor HAT è stato inserito direttamente sul connettore GPIO a 40 pin del Raspberry Pi, stabilendo il collegamento per i segnali d'interfaccia I2C (SDA e SCL), la massa comune e l'alimentazione logica a 3.3V.

L'alimentatore da banco esterno è stato connesso alla morsettiera dedicata ai motori dell'HAT. I carichi sono stati cablati come segue:

1. I due terminali del motore DC sono stati collegati alla morsettiera di uscita contrassegnata come **M1**.
2. Le due coppie di fasi del motore passo-passo bipolare sono state collegate ai blocchi terminali combinati **M1** e **M2**

4.2 Procedura di Controllo Software (Python 3)

Il controllo degli attuatori è stato eseguito in ambiente Linux tramite script in ambiente Python 3, sfruttando lo stack software composto dalle librerie `Adafruit_Blinka` (per la compatibilità `CircuitPython`) e `adafruit_motorkit`.

- **Caratterizzazione Motore DC:** Tramite l'inizializzazione dell'oggetto `kit = MotorKit()`, si è modificato progressivamente il valore dell'attributo di comando `kit.motor1.throttle` in un intervallo compreso tra 0.1 e 1.0 (corrispondente ad un Duty Cycle reale compreso tra il 10% e il 100%). Per ciascun incremento dello 0.1, si è acquisita la velocità di rotazione in RPM e la forma d'onda della tensione sul canale d'uscita.
- **Verifica del Motore Stepper:** È stata richiamata ciclicamente la funzione `kit.stepper1.onestep()` configurando l'argomento `style` nei quattro formati disponibili (`SINGLE`, `DOUBLE`, `INTERLEAVE`, `MICROSTEP`), misurando l'entità dello spostamento angolare risultante e verificandone l'andamento all'oscilloscopio.

5 Raccolta e Analisi dei Dati Sperimentali

5.1 Dati Rilevati sul Motore DC

5.2 Dati Rilevati sul Motore Stepper

6 Discussione dei Risultati

6.1 Comportamento Dinamico del Motore DC

Dall'analisi del grafico di linearità si evince che impostando un valore di *throttle* pari a 0.1 (tensione ai morsetti pari a 1.2 V), il motore non avvia la propria rotazione, facendo registrare una velocità pari a 0 RPM. Tale comportamento evidenzia sperimentalmente la presenza della **zona morta** dell'attuatore, determinata dalla necessità di accumulare una coppia sufficiente a vincere le forze d'attrito statico di primo distacco presenti sui cuscinetti.

Per valori di comando superiori o uguali a 0.2, la curva tensione-velocità assume un andamento prettamente lineare. Questo trend convalida sperimentalmente l'equazione semplificata di funzionamento a vuoto del motore DC ($V_a \approx K_e \cdot \omega$), confermando che il controllo di velocità in anello aperto tramite la regolazione del Duty Cycle del chip PCA9685 risulta efficace e stabile.

6.2 Valutazione delle Modalità di Stepping del Motore Passo-Passo

Le prove sperimentali condotte sul motore Stepper hanno evidenziato una netta correlazione tra lo stile di pilotaggio software impostato e le performance meccaniche dell'albero: * Nelle modalità SINGLE e DOUBLE, l'invio di una sequenza di 200 impulsi ha completato esattamente una rotazione completa di 360° , convalidando l'accuratezza dell'angolo di passo nominale di 1.8° . Il pilotaggio a doppia bobina (DOUBLE) ha evidenziato una maggiore rigidità e resistenza alla perdita del passo se sottoposto a carichi torsionali manuali esterni. * La modalità INTERLEAVE (Half-Step) ha richiesto l'esecuzione di 400 sotto-passi per completare un giro intero, confermando il dimezzamento dell'angolo di scomposizione spaziale a 0.9° . Dinamicamente si è osservata una riduzione drastica dello scottonamento tipico dei passi pieni. * La modalità MICROSTEP ha esibito la massima fluidità di rotazione, assimilabile a quella continua di un motore DC a spazzole, eliminando quasi del tutto le vibrazioni acustiche e i picchi di risonanza sull'albero.

7 Conclusioni

L'esperienza condotta ha consentito di testare con successo l'efficacia del controllo di attuatori elettromeccanici mediante un computer a scheda singola ad alte prestazioni (Raspberry Pi) accoppiato a un'architettura hardware dedicata basata sui chip specialistici TB6612 e PCA9685.

Il motore DC si è dimostrato ideale per applicazioni in cui è richiesta la regolazione continua della velocità tramite variazione del Duty Cycle, a patto di compensare via software la non-linearità introdotta dalla zona morta iniziale. Il motore passo-passo, di contro, ha confermato le sue eccellenti doti di posizionatore ad alta precisione in anello aperto; l'utilizzo delle funzionalità di

microstepping offerte dall'HAT Adafruit rappresenta la scelta ottimale qualora l'applicazione industriale richieda fluidità di movimento e assenza di vibrazioni meccaniche.